

Dokumentasi Proyek Haki

version

Candrasa

August 12, 2026

Contents

Welcome to Haki's Documentation!	1
Modules	1
S111_70_85_smart_grid_auto_iterasi_landmask module	1
DFSU -> S-111 HDF5 PIPELINE — v2.10 LANDMASK REFACTOR	1
s104_v2_5_landmask module	5
DFSU → S-104 HDF5 PIPELINE WITH INTEGRATED LAND MASKING	5
IHO S-104 Edition 2.0.0 DCF 2 WGS84 MLLW	5
Indices and tables	8
Index	9
Python Module Index	11

Welcome to Haki's Documentation!

Modules

S111_70_85_smart_grid_auto_iterasi_landmask module

DFSU -> S-111 HDF5 PIPELINE — v2.10 LANDMASK REFACTOR

IHO S-111 Edition 2.0.0 | DCF 2 | WGS84 Multi-Area Grid Support — MIKE FM Hydrodynamic Model

FIX v2.10 (perbaikan dari v2.9): [F40] `classify_grid_by_land_ratio()`: klasifikasi grid (darat/laut/campuran) [F41] `mask_extrapolation_boundary()`: BUG-U6 fix untuk boundary ekstrapolasi [F42] `generate_new_grid_names()`: rename grid laut secara berurutan (Grid01, Grid02, ...) [F43] `save_grid_mapping()`: simpan mapping nama lama->baru ke file JSON [F44] `process_one_timestep()`: terapkan `mask_extrapolation_boundary` untuk hapus data bbox terluar darat [F45] Main: filter grid berdasarkan klasifikasi, terapkan rename grid laut [F46] Grid-specific landmask: terapkan landmask untuk grid campuran, skip grid darat murni [F28] Grid-specific output subdirectory untuk menghindari tabrakan filename [F29] `configure_area_from_gridfile`: support single name atau list [F30] Refactor main() -> `process_single_grid()` untuk pipeline per-grid [F31] Per-gra land mask, adaptive regrid, dan grid properties =====

`S111_70_85_smart_grid_auto_iterasi_landmask.adaptive_grid_resolution`(bbox: list, dx: float, dy: float, n_timesteps: int, max_mb: float, dfsu_res_m: float = None) → tuple

`S111_70_85_smart_grid_auto_iterasi_landmask.apply_land_mask_to_h5`(h5_path: str, land_mask: numpy.ndarray, grid_properties: dict) → dict
[F18][F19] Terapkan land mask dengan validasi ukuran.

`S111_70_85_smart_grid_auto_iterasi_landmask.apply_land_mask_to_h5_multi`(h5_path: str, inst_idx: int, land_mask: numpy.ndarray, grid_properties: dict) → dict
[F35] Land mask untuk instance `SurfaceCurrent.<NN>` tertentu pada HDF5 multi-instance. NN diformat 2-digit zero-padded (01, 02, ..., 21).

`S111_70_85_smart_grid_auto_iterasi_landmask.build_bbox_mask`(elem_x, elem_y, layer_ids, unique_lids, bbox, dfsu_type)

`S111_70_85_smart_grid_auto_iterasi_landmask.build_day_blocks`(sorted_days: list, mode: str) → list

Kelompokkan daftar hari menjadi blok sesuai OUTPUT_MODE.

- “daily” -> 1 hari per blok : [[d1], [d2], ...]

- “3day” -> BLOCK_DAYS per blok: [[d1,d2,d3], [d4,d5,d6], ...]

Blok terakhir boleh berisi < BLOCK_DAYS hari (sisanya). Setiap blok menghasilkan SATU file HDF5 (semua timestep hari-hari dalam blok ditulis sebagai Group_001..Group_NNN berurutan).

`S111_70_85_smart_grid_auto_iterasi_landmask.build_grid_and_properties`(bbox, dx, dy)

`S111_70_85_smart_grid_auto_iterasi_landmask.build_land_mask`(shp_path: str, grid_properties: dict, land_union=None) → numpy.ndarray

[F16][F17][F20][F47][REFACTOR v3.0] Build land mask dari shapefile dengan GDAL rasterisasi optimal.

Perbaikan v3.0: - Filosofi baru: SEMUA grid diproses, landmask hanya menghapus sel darat - Hapus buffer negatif yang menyebabkan masking berlebihan - Validasi CRS shapefile sebelum proses - Optimasi memori dengan rasterisasi langsung tanpa meshgrid

[RENUMBER v3.1] Param opsional `land_union`: bila diberikan (hasil `load_land_union`, dibaca SEKALI untuk seluruh area gabungan), shapefile TIDAK dibaca ulang per-grid — mask dihitung langsung dari geometry via `shapely contains_xy` pada titik pusat sel grid. Orientasi flat 1D HARUS identik dengan jalur GDAL (row=lat menaik, col=lon menaik, ravel order="C") — lihat rekonstruksi `lon_arr/lat_arr` di bawah, yang mereproduksi PERSIS `np.arange(bbox[0], bbox[2], dx)` dari `build_grid_and_properties` (start & step sama -> hasil arange identik). Jika `land_union` None, perilaku SAMA PERSIS seperti sebelumnya (path GDAL asli, backward compatible, default).

`S111_70_85_smart_grid_auto_iterasi_landmask.build_s111_metadata`(datetime_issuance, geographic_id='Teluk Jakarta, Indonesia')

`S111_70_85_smart_grid_auto_iterasi_landmask.calculate_estimated_file_size` (nx: int, ny: int, n_groups: int, bytes_per_cell: int = 12) → float

Estimasi ukuran file HDF5 dalam MB (12 bytes/sel × 1.2 overhead × groups).

`S111_70_85_smart_grid_auto_iterasi_landmask.classify_and_filter_grids`

(grids_to_process: dict, output_dir: str) → tuple

[LMK-04] [F48] Generate rename mapping untuk SEMUA grid — TANPA filtering.

Perbaikan v2.12: - Hapus filtering grid berdasarkan klasifikasi darat/laut - Semua grid yang memiliki data arus DFSU diproses - Landmask diterapkan per-grid untuk menghapus sel darat saja

Parameters:

- **grids_to_process** – Dict {grid_name: {"bbox": [...], ...}}

- **output_dir** – Direktori output untuk menyimpan mapping file

Returns: (grids_filtered, grid_rename_mapping, grid_classifications) - grids_filtered: Dict {grid_name: grid_cfg} SEMUA grid (no filtering) - grid_rename_mapping: Dict {old_name: new_name} - grid_classifications: Dict {grid_name: "land"|"sea"|"mixed"}

`S111_70_85_smart_grid_auto_iterasi_landmask.classify_and_renumber_grids`

(grids_to_process: dict, land_union, elem_x=None, elem_y=None) → tuple

Klasifikasi tiap grid (PURE_LAND/MIXED/PURE_SEA/NO_DATA) dan penomoran ulang spasial (row=0 di selatan, col=0 di barat; urutan S->N lalu W->E).

Grid yang PURE_LAND (land_frac >= LAND_SKIP_THRESHOLD) atau NO_DATA (tidak ada elemen DFSU di dalam bbox, bila elem_x/elem_y diberikan) dikeluarkan (keep=False) dari output S-111 — tapi tetap dilaporkan di CSV/JSON mapping & PNG catalog untuk keperluan evaluasi.

Parameters:

- **grids_to_process** – dict {name: {"bbox": [minlon, minlat, maxlon, maxlat], ...}}

- **lon-first**. ((format S-111 parse_grid_file — bbox)

- **land_union** – shapely geometry (hasil load_land_union) atau None

- **grid** ((-> land_frac=0.0 untuk semua)

- **PURE_SEA**. (semua dianggap)

- **elem_x** – koordinat elemen DFSU (opsional). Jika None, n_elem

- **elem_y** – koordinat elemen DFSU (opsional). Jika None, n_elem

- **dihitung** (tidak)

Returns: (kept_ordered, mapping, classified) - kept_ordered: dict {old_name: cfg} — HANYA grid KEPT,urut new_num. - mapping: dict {old_name: new_name} — HANYA grid KEPT. - classified: list[dict] — semua grid (termasuk yang di-skip), keys: old_name, bbox, center_lon, center_lat, land_frac, n_elem, tile_class, keep, row, col, new_name, new_num.

`S111_70_85_smart_grid_auto_iterasi_landmask.configure_area_auto` (ds_open)

`S111_70_85_smart_grid_auto_iterasi_landmask.configure_area_from_gridfile`

(grid_file_path, selected_grid, dx, dy)

[F29] Support single name (str) atau list of names. Mengembalikan dict {name: bbox} agar caller bisa iterasi.

`S111_70_85_smart_grid_auto_iterasi_landmask.configure_area_from_list` (area_key: str) → tuple

`S111_70_85_smart_grid_auto_iterasi_landmask.configure_area_manual` (bbox, dx, dy)

`S111_70_85_smart_grid_auto_iterasi_landmask.detect_dfsu_resolution` (elem_x:

numpy.ndarray, elem_y: numpy.ndarray, n_samples: int = 500) → float

`S111_70_85_smart_grid_auto_iterasi_landmask.generate_new_grid_names` (valid_grids: list) → dict

Hasilkan mapping nama grid lama -> baru dengan urutan zero-padded.

Parameters: **valid_grids** – List nama grid yang valid (sudah difilter,urut)

Returns: nama_baru, misal {"Grid1": "Grid01", "Grid5": "Grid02"}

Return type: Dict {nama_lama

`S111_70_85_smart_grid_auto_iterasi_landmask.load_land_union` (shp_path: str, full_bbox: tuple, buffer_deg: float = 0.0)
Baca shapefile daratan SEKALI (clipped ke full_bbox gabungan semua grid), lalu `union_all()` + buffer. Dipakai agar N grid tidak masing-masing membaca shapefile sendiri-sendiri (mahal untuk ratusan grid).

Parameters:

- **shp_path** – path shapefile.
- **full_bbox** – (minlon, minlat, maxlon, maxlat) — bbox gabungan seluruh grid.
- **buffer_deg** – buffer dalam derajat (default LAND_BUFFER_DEG).

Returns: shapely geometry (union, sudah di-buffer) atau None jika kosong.

`S111_70_85_smart_grid_auto_iterasi_landmask.main()`

`S111_70_85_smart_grid_auto_iterasi_landmask.mask_extrapolation_boundary` (u_grid: numpy.ndarray, v_grid: numpy.ndarray, grid_xx: numpy.ndarray, grid_yy: numpy.ndarray, elem_x_surf: numpy.ndarray, elem_y_surf: numpy.ndarray) → tuple
[BUG-U6] Set NODATA untuk sel grid yang terlalu jauh dari elemen DFSU terdekat. Ini mengatasi boundary extrapolation yang menghasilkan data arus palsu di area darat di luar jangkauan model.

Parameters:

- **u_grid** – Grid 2D hasil regrid (float32)
- **v_grid** – Grid 2D hasil regrid (float32)
- **grid_xx** – Meshgrid koordinat grid (2D)
- **grid_yy** – Meshgrid koordinat grid (2D)
- **elem_x_surf** – Koordinat elemen DFSU dalam BBOX
- **elem_y_surf** – Koordinat elemen DFSU dalam BBOX

Returns: (u_grid, v_grid) dengan boundary extrapolation di-mask NODATA

`S111_70_85_smart_grid_auto_iterasi_landmask.open_and_detect_dfsu` (dfs_path: str)

`S111_70_85_smart_grid_auto_iterasi_landmask.parse_grid_file` (grid_file_path: str) → dict

`S111_70_85_smart_grid_auto_iterasi_landmask.plot_grid_catalog` (classified: list, land_shp: str, out_png: str, dpi: int = 200) → str

Render peta katalog grid: poligon daratan sebagai latar, kotak grid berwarna per kelas, nomor baru untuk grid yang kept.

`S111_70_85_smart_grid_auto_iterasi_landmask.prepare_grid_context` (grid_name, grid_cfg, ds_open, dfsu_type, elem_x, elem_y, layer_ids, unique_lids, land_union=None)

[F33] Siapkan konteks lengkap untuk satu grid TANPA memproses timestep. Mengembalikan dict dengan: bbox, bbox_mask_2d, elem_x_surf, elem_y_surf, grid_xx, grid_yy, grid_properties, land_mask, status.

Dipakai oleh mode multi-instance (semua grid dievaluasi dulu, lalu instance dibuat satu per satu di HDF5 yang sama).

`S111_70_85_smart_grid_auto_iterasi_landmask.process_daily` (date_key, day_timesteps, ds_open, dfsu_type, bbox_mask_2d, elem_x_surf, elem_y_surf, grid_xx, grid_yy, grid_properties, metadata, output_dir, prefix='111_70_85_smart_grid_', land_mask=None, date_key_end=None)

`S111_70_85_smart_grid_auto_iterasi_landmask.process_daily_multi_instance` (date_key, day_timesteps, ds_open, dfsu_type, grid_contexts, metadata, output_dir, prefix='111_70_85_smart_grid_')

[F33] Tulis SEMUA grid sebagai instance SurfaceCurrent.01 .. SurfaceCurrent.N dalam satu HDF5 (mode DCF2 multi-instance sesuai S-111 spec).

Setiap grid -> `add_surface_current_instance` (instance baru) lalu untuk setiap timestep di hari tersebut -> `add_data_from_arrays` (group baru pada instance terakhir).

Parameters:

- **date_key** – datetime.date untuk filename & logging
- **day_timesteps** – list timestep pada hari tersebut
- **ds_open** – mikeio.Dataset
- **dfs_type** – “2D” / “3D”
- **grid_contexts** – list of dict dari prepare_grid_context()
- **metadata** – dict metadata S-111 (root)
- **output_dir** – direktori output utama
- **prefix** – prefix filename

Returns: (output_h5, n_grid_written, n_groups_written)

`S111_70_85_smart_grid_auto_iterasi_landmask.process_hourly(t, ds_open, dfs_type, bbox_mask_2d, elem_x_surf, elem_y_surf, grid_xx, grid_yy, grid_properties, metadata, output_dir, prefix='111_70_85_smart_grid_', land_mask=None)`

`S111_70_85_smart_grid_auto_iterasi_landmask.process_one_timestep(t, ds_open, dfs_type, bbox_mask_2d, elem_x_surf, elem_y_surf, grid_xx, grid_yy)`

`S111_70_85_smart_grid_auto_iterasi_landmask.process_single_grid(grid_name, grid_cfg, ds_open, dfs_type, elem_x, elem_y, layer_ids, unique_lids, daily_groups, sorted_days, output_dir_base, grid_rename_mapping=None, land_union=None)`

[F30] Pipeline lengkap untuk memproses satu grid:

1. BBOX mask
2. Deteksi resolusi DFSU
3. Adaptive regrid
4. Build grid
5. Build land mask
6. Proses semua timestep (daily/hourly)

Parameters:

- **grid_name** – Nama grid (untuk logging & output naming)
- **grid_cfg** – Dict {"bbox": [...]} atau object serupa
- **ds_open** – Dataset MIKE IO yang sudah terbuka
- **dfs_type** – “2D” atau “3D”
- **elem_x** – Koordinat elemen DFSU
- **elem_y** – Koordinat elemen DFSU
- **layer_ids** – Layer IDs (None untuk 2D)
- **unique_lids** – Unique layer IDs (None untuk 2D)
- **daily_groups** – Dict {date: [timesteps]}
- **sorted_days** – List date yang sudah difilter
- **output_dir_base** – Direktori output utama

Returns: List path file HDF5 yang dihasilkan

`S111_70_85_smart_grid_auto_iterasi_landmask.regrid_to_s111(elem_x, elem_y, values, grid_xx, grid_yy)`

`S111_70_85_smart_grid_auto_iterasi_landmask.regrid_uv_to_s111(elem_x: numpy.ndarray, elem_y: numpy.ndarray, raw_u: numpy.ndarray, raw_v: numpy.ndarray, grid_xx: numpy.ndarray, grid_yy: numpy.ndarray) → tuple[numpy.ndarray, numpy.ndarray]`

`S111_70_85_smart_grid_auto_iterasi_landmask.reopen_dfsu(old_ds_open=None, dfsu_path: str = 'D:\\Clivar S-100 Benoa\\S100\\Lat_S100series (1)\\Lat_S100series\\HD_PLI_78.dfsu')`

[MEMFIX] Buka ulang handle DFSU setelah `ds_open.read()` gagal (biasanya dipicu tekanan memori) dan mem-poison state internal reader mikeio, menyebabkan kaskade DFS error code 2019 pada timestep berikutnya. Hanya handle dataset yang di-refresh — geometri (`elem_x/elem_y/layer_ids`) tidak berubah sehingga tidak perlu dihitung ulang oleh caller.

`S111_70_85_smart_grid_auto_iterasi_landmask.safe_close_and_create_s111` (output_h5: str, dcf: int = 2)

`S111_70_85_smart_grid_auto_iterasi_landmask.safe_close_and_delete` (output_h5: str)
Close HDF5 file handle and delete file if no valid data written.

`S111_70_85_smart_grid_auto_iterasi_landmask.save_grid_mapping` (mapping: dict, output_path: str, grids: dict = None)
Simpan mapping grid lama -> baru ke file CSV.

Parameters:

- **mapping** – Dict {old_name: new_name}
- **output_path** – Path tujuan file CSV
- **grids** – Dict opsional {grid_name: {"bbox": [...]}} untuk menyertakan koordinat BBOX di CSV.

Format: old_name,new_name,minx,miny,maxx,maxy

`S111_70_85_smart_grid_auto_iterasi_landmask.save_grid_mapping_csv` (classified: list, out_csv: str) → str
Tulis CSV mapping old_name -> new_name (+ row/col/class/land_frac/dll).

`S111_70_85_smart_grid_auto_iterasi_landmask.save_grid_mapping_json` (mapping: dict, classified: list, json_path: str) → str
[RENUMBER v3.1] Simpan mapping lengkap old_name -> detail klasifikasi + new_name ke file JSON (mengaktifkan GRID_MAPPING_FILE yang sebelumnya didefinisikan tapi tidak pernah ditulis).

`S111_70_85_smart_grid_auto_iterasi_landmask.uv_to_speed_direction` (u_grid: numpy.ndarray, v_grid: numpy.ndarray) → tuple[numpy.ndarray, numpy.ndarray]

`S111_70_85_smart_grid_auto_iterasi_landmask.validate_land_masked_h5` (h5_path: str, land_mask: numpy.ndarray) → bool

s104_v2_5_landmask module

DFSU → S-104 HDF5 PIPELINE WITH INTEGRATED LAND MASKING

IHO S-104 Edition 2.0.0 | DCF 2 | WGS84 | MLLW

PIPELINE TERINTEGRASI:

1. Konversi DFSU → Grid S-104 (regrid + datum conversion + trend)
2. Build Land Mask dari Shapefile (OSM/Custom)
3. Apply Land Mask ke semua timestep
4. Output: HDF5 S-104 masked (daily/hourly/3day/single)

Struktur S-104 DCF=2 (Regular Grid) yang dihasilkan:

/WaterLevel/WaterLevel.01

attrs: `gridOriginLongitude`, `gridOriginLatitude`,
`gridSpacingLongitudinal`, `gridSpacingLatitudinal`, `numPointsLongitudinal`, `numPointsLatitudinal`

/Group_001/

attr: `timePoint` (YYYYMMDDTHHMMSSZ) dataset: values — 2D compound array [NY, NX] compound fields: `waterLevelHeight` (f32), `waterLevelTrend` (uint8) fillValue height: -9999.0 fillValue trend: 0 (Unknown)

Fitur:

- Single-pass processing (tidak ada file intermediate)
- Land mask reusable (disimpan sebagai dataset di HDF5)
- Auto-patch atribut grid
- Multiple output modes: daily | hourly | 3day | single
- Validasi hasil masking
- Metadata IHO S-104 Ed. 2.0.0 compliant

Dependencies:

mikeio, s100py, scipy, numpy, h5py, geopandas, shapely

`s104_v2_5_landmask.apply_land_mask_to_data` (wl_grid: numpy.ndarray, wl_trend: numpy.ndarray, land_mask: numpy.ndarray) → tuple

Apply land mask ke grid water level dan trend. waterLevelHeight → NODATA_WL (-9999.0) waterLevelTrend → NODATA_TREND (0 = Unknown)

`s104_v2_5_landmask.build_bbox_mask` (elem_x, elem_y, layer_ids, unique_lids, bbox, dfsu_type)
Filter elemen DFSU yang berada dalam BBOX.

`s104_v2_5_landmask.build_grid_and_properties` (bbox, dx, dy)
Bangun grid terstruktur S-104 dari BBOX dan resolusi.

`s104_v2_5_landmask.build_land_mask` (shp_path: str, lon_arr: numpy.ndarray, lat_arr: numpy.ndarray, land_union=None) → tuple
Build land mask dari shapefile polygon daratan.

Parameters: **land_union** – jika diberikan (hasil load_land_union), shapefile TIDAK dibaca ulang — langsung dipakai untuk point-in-polygon test. Jika None, perilaku SAMA PERSIS seperti sebelumnya (baca shapefile per-panggilan, backward compatible).

Returns: Boolean array (NY, NX), True = daratan mask_info: Dict info masking

Return type: land_mask

`s104_v2_5_landmask.build_s104_metadata` (geographic_id='Selat Lombok, Indonesia')
Build metadata IHO S-104 Ed. 2.0.0.

`s104_v2_5_landmask.build_split_plan` (n_groups: int, mode: str) → list
Bangun rencana split output.

`s104_v2_5_landmask.classify_and_renumber_tiles` (areas: list, land_union, elem_x=None, elem_y=None) → list
Klasifikasi tiap tile (PURE_LAND/MIXED/PURE_SEA/NO_DATA) dan penomoran ulang spasial (row=0 di selatan, col=0 di barat; urutan S→N lalu W→E).
Tile yang PURE_LAND (land_frac >= LAND_SKIP_THRESHOLD) atau NO_DATA (tidak ada elemen DFSU di dalam bbox, bila elem_x/elem_y diberikan) dikeluarkan (keep=False) dari output S-104 — tapi tetap dilaporkan di CSV mapping & PNG catalog untuk keperluan evaluasi.

Parameters:

- **areas** – list of dict dari parse_area_list (name, min_lat, min_lon, max_lat, max_lon).
- **land_union** – shapely geometry (hasil load_land_union) atau None (→ land_frac=0.0 untuk semua tile, semua dianggap PURE_SEA).
- **elem_x** – koordinat elemen DFSU (opsional). Jika None, n_elem tidak dihitung (None) dan tidak dipakai untuk exclude.
- **elem_y** – koordinat elemen DFSU (opsional). Jika None, n_elem tidak dihitung (None) dan tidak dipakai untuk exclude.

Returns: name, bbox, center_lon, center_lat, land_frac, n_elem, tile_class, keep, row, col, new_name, new_num.

Return type: list[dict] — satu entry per tile (termasuk yang di-skip), keys

`s104_v2_5_landmask.compute_masked_stats` (wl_grid: numpy.ndarray, land_mask: numpy.ndarray) → dict
Hitung statistik untuk sel perairan valid saja.

`s104_v2_5_landmask.compute_wl_trend(wl_current, wl_prev, threshold=0.2)`

Hitung trend water level per S-104 spec: 1=Decreasing, 2=Increasing, 3=Steady, 0=Unknown (NODATA)

`s104_v2_5_landmask.create_s104_hdf5(output_h5: str, dcf: int = 2)`

Create file S-104 HDF5 baru menggunakan s100py.

`s104_v2_5_landmask.load_land_union(shp_path: str, full_bbox: tuple, buffer_deg: float = -5e-05)`

Baca shapefile daratan SEKALI (clipped ke full_bbox gabungan semua tile), lalu `union_all()` + buffer. Dipakai agar 154 tile tidak masing-masing membaca shapefile sendiri-sendiri (mahal untuk ratusan tile).

Parameters:

- **shp_path** – path shapefile.
- **full_bbox** – (minlon, minlat, maxlon, maxlat) — bbox gabungan seluruh tile.
- **buffer_deg** – buffer dalam derajat (default LAND_BUFFER_DEG).

Returns: shapely geometry (union, sudah di-buffer) atau None jika kosong.

`s104_v2_5_landmask.main()`

Entry point utama dengan mode selection.

`s104_v2_5_landmask.open_and_detect_dfsu(dfsu_path: str)`

Buka file DFSU dan deteksi tipe (2D/3D).

`s104_v2_5_landmask.parse_area_list(path: str) → list`

Parse area_list.txt (21 grid) menjadi list of dict.

Format tiap baris (CSV tanpa header):

Name,min_lat,min_lon,max_lat,max_lon

Returns: name, min_lat, min_lon, max_lat, max_lon.

Return type: list[dict] keys

`s104_v2_5_landmask.plot_tile_catalog(classified: list, land_shp: str, out_png: str, dpi: int = 200) → str`

Render peta katalog tile: poligon daratan sebagai latar, kotak tile berwarna per kelas, nomor baru untuk tile yang kept.

`s104_v2_5_landmask.process_one_day_full(date_key, day_timesteps, ds_open, dfsu_type, bbox_mask_2d, elem_x_surf, elem_y_surf, grid_xx, grid_yy, grid_properties, metadata, output_dir, land_mask, mask_info, wl_prev_carry=None, prefix='104_70_85_', wl_cache=None, time_to_idx=None, bbox_elem_idx=None)`

Proses satu hari: DFSU → Regrid → Mask → Write HDF5. Land mask di-apply langsung saat write (single-pass).

Args: `wl_cache`, `time_to_idx`, `bbox_elem_idx`: bila `wl_cache` diberikan (preload RAM, lihat `run_multi_grid_pipeline`), timestep dibaca dari array in-memory alih-alih `ds_open.read()/isel()` per timestep — jauh lebih cepat untuk ratusan tile. Bila `wl_cache` None, perilaku SAMA PERSIS seperti sebelumnya (baca DFSU langsung per timestep).

`s104_v2_5_landmask.regrid_to_s104(elem_x, elem_y, values, grid_xx, grid_yy)`

Interpolasi dari mesh tidak terstruktur ke grid terstruktur.

`s104_v2_5_landmask.run_catalog_only()`

Mode `catalog_only`: `parse_area_list` → `load_land_union` → klasifikasi + penomoran ulang tile → PNG catalog + CSV mapping. TIDAK butuh DFSU/s100py — hanya geopandas+shapely (+matplotlib opsional untuk PNG).

Berguna untuk evaluasi cepat tata-letak tile & land masking sebelum menjalankan pipeline penuh (yang butuh DFSU dan berjalan jauh lebih lama).

`s104_v2_5_landmask.run_full_pipeline()`

Pipeline lengkap: DFSU → S-104 → Land Mask (single-pass).

`s104_v2_5_landmask.run_mask_only(input_h5: str, output_dir: str, land_shp: str)`

Mode `mask-only`: Baca HDF5 S-104 existing → Apply land mask → Split output.

`s104_v2_5_landmask.run_multi_grid_pipeline()`

Pipeline MULTI-GRID: baca N grid dari `area_list.txt` → hasilkan HDF5 TERPISAH per grid (satu subfolder per grid).

Alur per-grid IDENTIK dengan `run_full_pipeline` (DFSU→regrid→datum→trend→ land mask→write, single-pass). Yang berbeda: - DFSU dibuka SEKALI lalu di-reuse untuk semua grid (efisiensi). - Land geometry (shapefile) dibaca SEKALI via `load_land_union`, reuse

untuk `build_land_mask` tiap tile (bukan baca shapefile per-tile).

- Tile diklasifikasi (PURE_LAND/MIXED/PURE_SEA/NO_DATA) & dinomori ulang secara spasial (S→N, W→E); tile PURE_LAND/NO_DATA dilewati.
- WL time-series di-preload SEKALI ke RAM (bila ukuran wajar) agar tidak ada pembacaan DFSU berulang per tile.
- BBOX, grid, dan land mask tetap dibangun ulang per tile (murah).
- Output ditulis ke `OUTPUT_DIR/<new_name>/` dengan prefix per tile.

`s104_v2_5_landmask.safe_close_hdf5` (output_h5: str)

Force close semua handle HDF5 yang terbuka untuk file tertentu.

`s104_v2_5_landmask.save_tile_mapping_csv` (classified: list, out_csv: str) → str

Tulis CSV mapping old_name → new_name (+ row/col/class/land_frac/dll).

`s104_v2_5_landmask.setup_logging` (output_dir: str)

Setup logging ke file dan console dengan UTF-8.

`s104_v2_5_landmask.validate_dependencies` (mode: str) → bool

Validasi dependensi berdasarkan mode operasi.

`s104_v2_5_landmask.validate_masked_h5` (h5_path: str, land_mask: numpy.ndarray) → bool

Validasi file HDF5 hasil masking.

`s104_v2_5_landmask.write_masked_hdf5_manual` (src_h5_path: str, dst_h5_path: str, land_mask: numpy.ndarray, split_indices: list, mask_info: dict, mode: str) → str

Salin struktur HDF5 dan apply land mask untuk subset groups. Compound dtype: waterLevelHeight (f32) + waterLevelTrend (uint8)

Indices and tables

- **genindex**
- **modindex**
- **search**

`s104_v2_5_landmask` S111_70_85_smart_grid_auto_iterasi_landmask

Index

A

`adaptive_grid_resolution()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`apply_land_mask_to_data()` (in module `s104_v2_5_landmask`)

`apply_land_mask_to_h5()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`apply_land_mask_to_h5_multi()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

B

`build_bbox_mask()` (in module `s104_v2_5_landmask`)

(in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`build_day_blocks()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`build_grid_and_properties()` (in module `s104_v2_5_landmask`)

(in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`build_land_mask()` (in module `s104_v2_5_landmask`)

(in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`build_s104_metadata()` (in module `s104_v2_5_landmask`)

`build_s111_metadata()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`build_split_plan()` (in module `s104_v2_5_landmask`)

C

`calculate_estimated_file_size()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`classify_and_filter_grids()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`classify_and_renumber_grids()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`classify_and_renumber_tiles()` (in module `s104_v2_5_landmask`)

`compute_masked_stats()` (in module `s104_v2_5_landmask`)

`compute_wl_trend()` (in module `s104_v2_5_landmask`)

`configure_area_auto()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`configure_area_from_gridfile()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`configure_area_from_list()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`configure_area_manual()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`create_s104_hdf5()` (in module `s104_v2_5_landmask`)

D

`detect_dfsu_resolution()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

G

`generate_new_grid_names()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

L

`load_land_union()` (in module `s104_v2_5_landmask`)

(in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

M

`main()` (in module `s104_v2_5_landmask`)

(in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`mask_extrapolation_boundary()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

module

- `s104_v2_5_landmask`
- `S111_70_85_smart_grid_auto_iterasi_landmask`

O

`open_and_detect_dfsu()` (in module `s104_v2_5_landmask`)

(in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

P

`parse_area_list()` (in module `s104_v2_5_landmask`)

`parse_grid_file()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`plot_grid_catalog()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`plot_tile_catalog()` (in module `s104_v2_5_landmask`)

`prepare_grid_context()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`process_daily()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

`process_daily_multi_instance()` (in module `S111_70_85_smart_grid_auto_iterasi_landmask`)

process_hourly() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

process_one_day_full() (in module s104_v2_5_landmask)

process_one_timestep() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

process_single_grid() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

R

regrid_to_s104() (in module s104_v2_5_landmask)

regrid_to_s111() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

regrid_uv_to_s111() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

reopen_dfsu() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

run_catalog_only() (in module s104_v2_5_landmask)

run_full_pipeline() (in module s104_v2_5_landmask)

run_mask_only() (in module s104_v2_5_landmask)

run_multi_grid_pipeline() (in module s104_v2_5_landmask)

S

s104_v2_5_landmask

module

S111_70_85_smart_grid_auto_iterasi_landmask

module

safe_close_and_create_s111() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

safe_close_and_delete() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

safe_close_hdf5() (in module s104_v2_5_landmask)

save_grid_mapping() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

save_grid_mapping_csv() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

save_grid_mapping_json() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

save_tile_mapping_csv() (in module s104_v2_5_landmask)

setup_logging() (in module s104_v2_5_landmask)

U

uv_to_speed_direction() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

V

validate_dependencies() (in module s104_v2_5_landmask)

validate_land_masked_h5() (in module S111_70_85_smart_grid_auto_iterasi_landmask)

validate_masked_h5() (in module s104_v2_5_landmask)

W

write_masked_hdf5_manual() (in module s104_v2_5_landmask)

Python Module Index

s

[s104_v2_5_landmask](#)

[S111_70_85_smart_grid_auto_iterasi_landmask](#)